

课程笔记

Agentic AI 时代的操作系统课：绪论

南京大学 操作系统原理 2026 春季学期 第 01 讲

Claude Code Notes

2026 年 3 月 24 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）

发布日期：2026 年春季学期

视频时长：1 小时 36 分钟

视频链接：<https://www.bilibili.com/video/BV1opAfzpEf9>

目录

1 引言	2
1.1 讲者简介	2
1.2 课程信息	3
1.3 本章小结	4
2 AI 时代还学操作系统吗？	4
2.1 AC 05 年：模型能力的爆发	4
2.2 从算法时代到 AI 时代：范式转移	5
2.3 学 CS 还有意义吗？	6
2.4 “操作系统”是你的最后一门编程课	7
2.5 本章小结	7
3 什么是操作系统	7
3.1 操作系统：不需要定义	8
3.2 复习：理解计算机硬件（电路）	8
3.3 Content as Code：今年课程的革新	9
3.4 复习：理解计算机软件（程序）	10
3.5 本章小结	11
4 操作系统的发展历史	11
4.1 1940s：电子计算机的黎明	11
4.2 1950s–1960s：Fortran 与早期软件	12
4.3 1960s–1970s：集成电路革命	13
4.4 1970s+：UNIX 与现代操作系统生态	14
4.5 本章小结	14
5 怎么学？	15
5.1 找到乐趣和动机	15
5.2 AIGC Policy	15
5.3 本章小结	16
5.4 拓展阅读	16
6 总结与延伸	16
6.1 讲者的核心信息	16
6.2 结构化提炼	17
6.3 跨章节关联	17
6.4 开放问题与下一步	17

1 引言

本讲是南京大学 2026 年春季学期《操作系统原理》课程的绪论。主讲人蒋炎岩 (jyy) 提出了一个核心主张：在 Agentic AI 时代，操作系统课程不仅没有过时，反而变得更加重要——因为操作系统正是支撑一切 AI Agent 运行的底层基础设施。

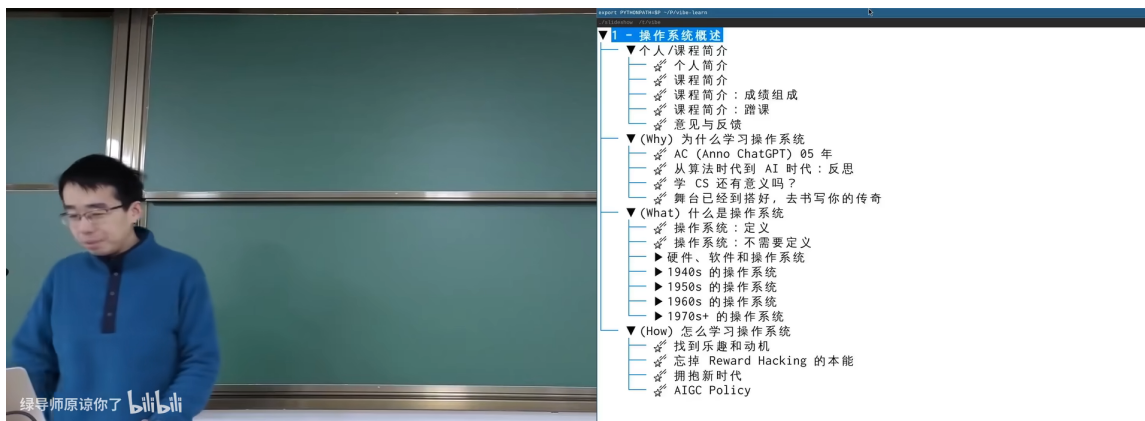


图 1: 课程大纲与本讲结构¹

本讲覆盖五个主题：

1. 个人/课程简介
2. (Why) 为什么在 AI 时代还要学操作系统
3. (What) 什么是操作系统——不需要定义
4. (History) 操作系统的发展历史
5. (How) 怎么学

1.1 讲者简介

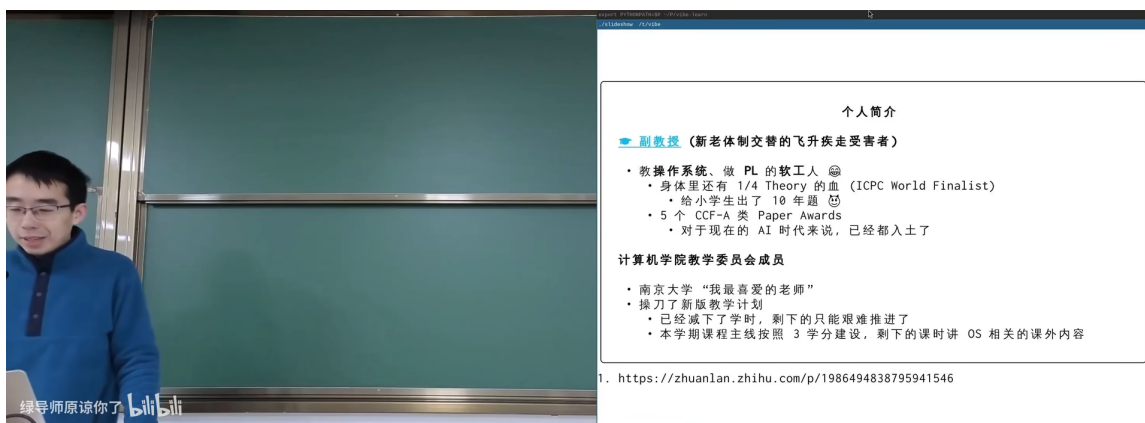


图 2: 蒋炎岩个人简介²

¹视频画面时间区间：00:00:15–00:00:45。

²视频画面时间区间：00:01:00–00:01:30。

蒋炎岩 (jyy)，南京大学计算机学院教师，研究方向为操作系统与程序设计语言 (PL)。关键履历：

- ICPC World Finalist (“身体里还有 1/4 Theory 的血”)
- 计算机领域深耕 18 年
- 5 篇 CCF-A 类论文，Paper Award 获得者
- 南京大学计算机学院教学委员会成员
- 课程已讲授多年，被誉为“现象级公开课”

jyy 的教学理念

jyy 自称是“新龙珠交替的飞升派金仙受害者”，强调对于 AI 时代来说，Paper Award 已经都入土了。他的教学理念可以概括为：用最前沿的工具和视角重新审视经典的操作系统知识。

1.2 课程信息

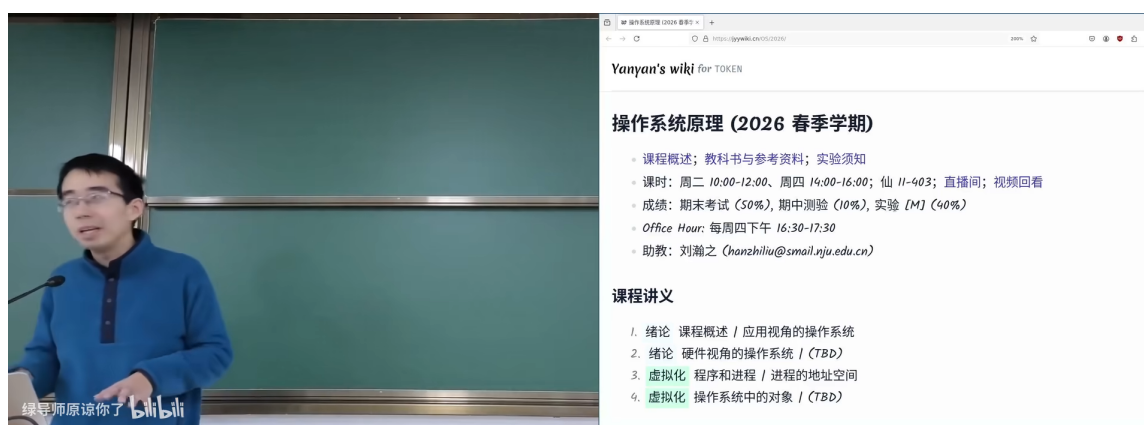


图 3: 课程 Wiki 页面：操作系统原理 2026 春季学期³

- **课时**：周二 10:00–12:00，周四 14:00–16:00，仙 II-648
- **直播**：程期回看
- **成绩**：期末考试 52%，期中测验 10%，实验 M1 10%
- **Office Hour**：每周四下午 16:30–17:30
- **课程讲义**：绪论 → 课程概述 → 应用视角的操作系统 → 虚拟化（硬件视角和操作系统 / TBD）→ 并发（程序和进程 / 进程的地址空间 / TBD）→ 持久化（操作系统中的对象 / TBD）

³视频画面时间区间：00:04:45–00:05:15。

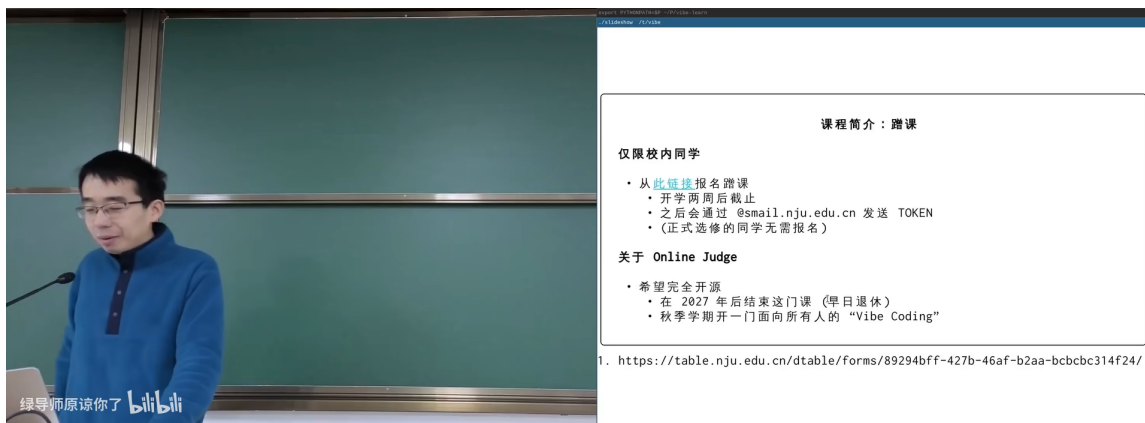


图 4: 蹭课与 Online Judge 政策⁴

关于 Online Judge

今年的 OJ 将在 2027 年后自由开放。jyy 将其描述为“秋季学期开了一门面向所有人的 Vibe Coding 课”，暗示 OJ 的设计会深度结合 AI 辅助编程。

1.3 本章小结

本节建立了课程的基本框架：一门由深耕操作系统 18 年的研究者讲授的课程，目标不是让学生记住 API 列表，而是理解计算机系统的“高光时刻”。

2 AI 时代还学操作系统吗？

这是本讲最具思辨性的章节。jyy 直面了一个尖锐的问题：当 AI 已经能写代码、能通过考试、甚至能拿 ICPC World Finals 金牌时，学操作系统还有意义吗？

2.1 AC 05 年：模型能力的爆发

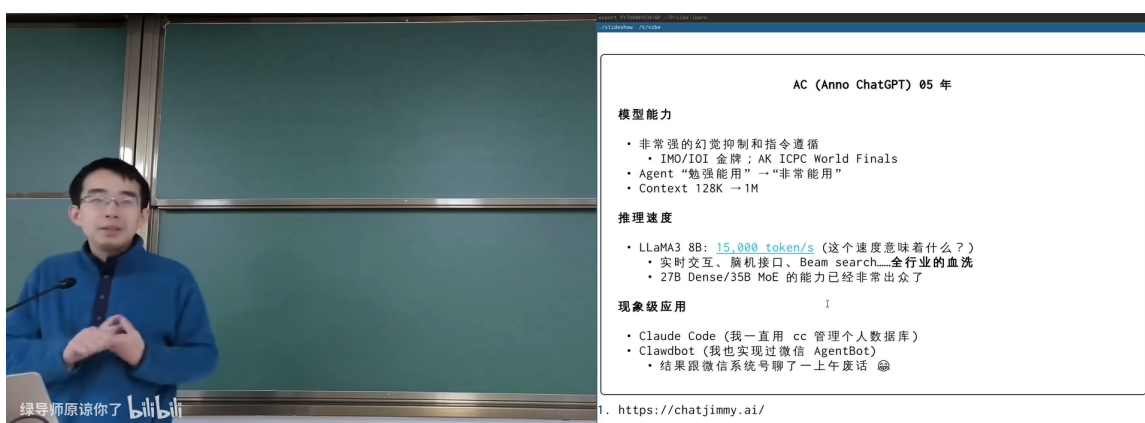


图 5: AC (Anno ChatGPT) 05 年：AI 能力的三个维度⁵

⁴视频画面时间区间：00:08:30–00:09:00。

⁵视频画面时间区间：00:13:15–00:14:00。

jyy 将当前时代称为“AC 05 年”（Anno ChatGPT，即 ChatGPT 纪元第 5 年），从三个维度描绘 AI 能力的现状：

1. 模型能力

- 竞赛：IMO/IOI 金牌，AK ICPC World Finals
- Agent 能力：“智能溢出”——不再仅是 token 生成器
- 上下文窗口：从 128K 扩展到 1M tokens

2. 推理速度

- LLaMA3 88B：15,000 tokens/s（“这个速度意味着什么？”）
- 实时交互、随机接口、Beam Search 已经成为全行业的血液
- 270 Dense/350 MoE 的推力已经非常高了

3. 极高价值应用

- Claude Code：用一套 CLI 管理个人数据和代码
- Clawdbot：现代实现迭代 AgentBot
- 结果震撼传统程序员——“一个下午搞定了一千个 issue”

对“AI 万能论”的警醒

jyy 并非在鼓吹 AI 万能。他在后续讨论中指出，AI 的能力在 **well-formed** 问题上表现出色，但复杂系统的灰度实现——那些需要深度理解抽象层设计的问题——仍然是人类工程师的核心价值所在。

2.2 从算法时代到 AI 时代：范式转移

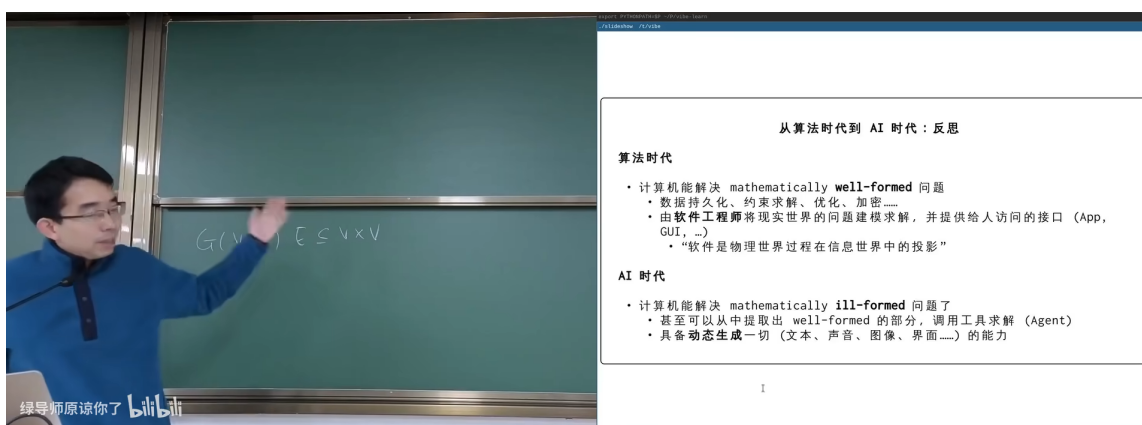


图 6: 算法时代 vs. AI 时代的根本区别⁶

⁶视频画面时间区间：00:25:30–00:26:15。

两个时代的核心差异

算法时代：计算机解决 *mathematically well-formed* 问题——数据分析、内存管理、优化、压缩……本质上是把人类难以触及的复杂问题通过编程接口（App、CLI……）暴露给用户。“软件是物理世界某些过程在信息世界中的投影。”

AI 时代：计算机开始能处理 *mathematically ill-formed* 问题——首先从中提取出 well-formed 的部分，调用工具来解（Agent）；同时具有融态生成一切的能力（文本、声音、图像、音频……）。

2.3 学 CS 还有意义吗？

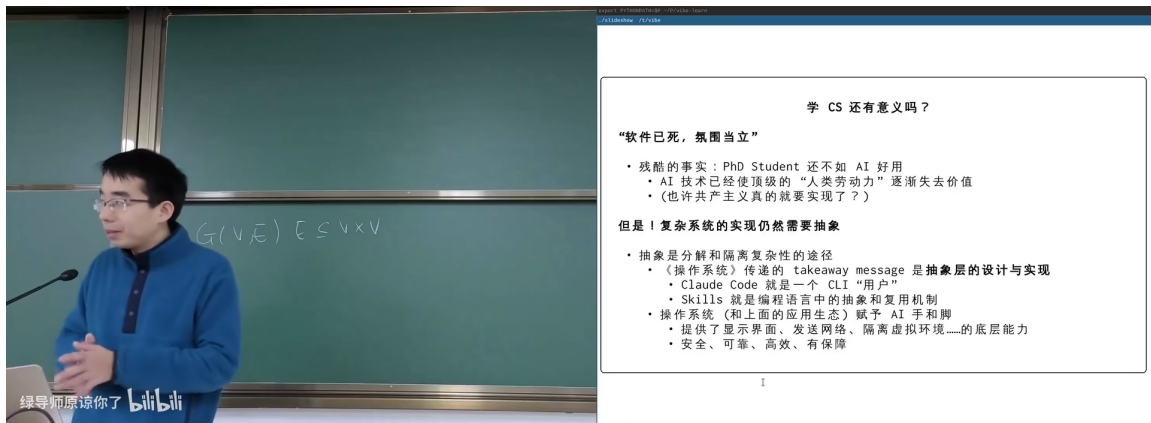


图 7: “软件已死，概率性立”——一个挑衅性的命题⁷

jyy 直面了最尖锐的质疑：“PhD Student 还不如 AI 好用”、“AI 技术已经使得很多人类劳动力岗位失去价值”。甚至引述了“但共产主义真的到来了吗？”这样的反问。

然而，他的回答是肯定的——**复杂系统的灰度实现仍然极其重要：**

- 抽象层的设计是核心：takeaway message 是“抽象层的设计与实现”
- 具体例子：Claude Code 扩展 → CLI → 用户界面；Skill 系统支持 API 集成
- 操作系统正是这种抽象层设计的典范

⁷视频画面时间区间：00:30:00–00:30:45。

2.4 “操作系统”是你的最后一门编程课

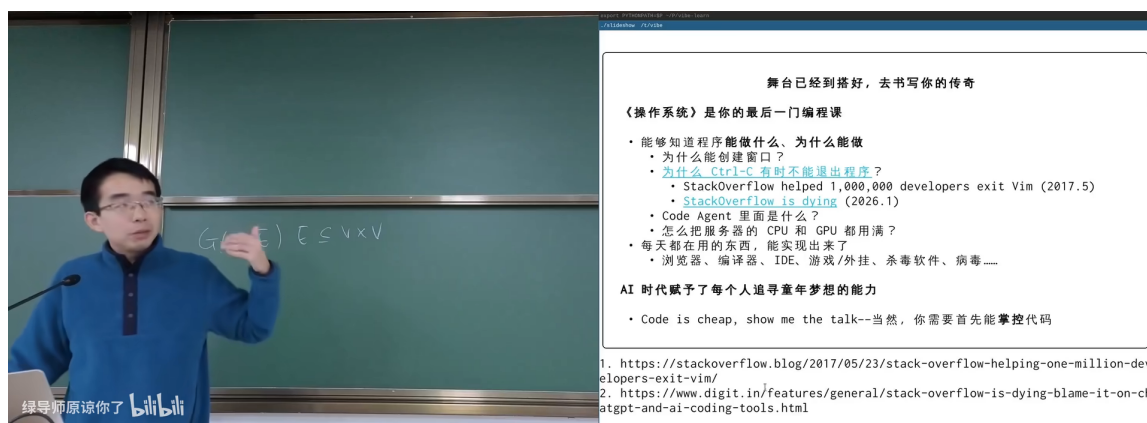


图 8: Code is cheap, show me the talk⁸

从“Talk is cheap”到“Code is cheap”

传统软件工程中 Linus Torvalds 的名言是“Talk is cheap, show me the code”。jyy 认为在 AI 时代这个命题被反转了：**Code is cheap, show me the talk**——代码可以由 AI 生成，但理解为什么需要这段代码、它在系统中扮演什么角色、抽象层如何设计，这才是人类的核心能力。

jyy 的核心论点：

- 编译器和进程做什么？理解这些是根本
- AI 时代赋予了每个人运算和梦想的能力
- 你需要首先能知道要做什么（“talk”），AI 帮你实现代码（“code”）
- 操作系统课程教你的正是“talk”——如何理解和设计系统

2.5 本章小结

AI 时代不是让操作系统知识过时，而是让它从“写代码的技能”升级为“理解系统的智慧”。关键洞察：**抽象层设计**是 AI 无法替代的核心能力，而操作系统是学习抽象层设计的最佳素材。

3 什么是操作系统

jyy 用一个反直觉的方式开始这个章节：**操作系统不需要定义**。

⁸视频画面时间区间：00:35:30-00:36:30。

3.1 操作系统：不需要定义

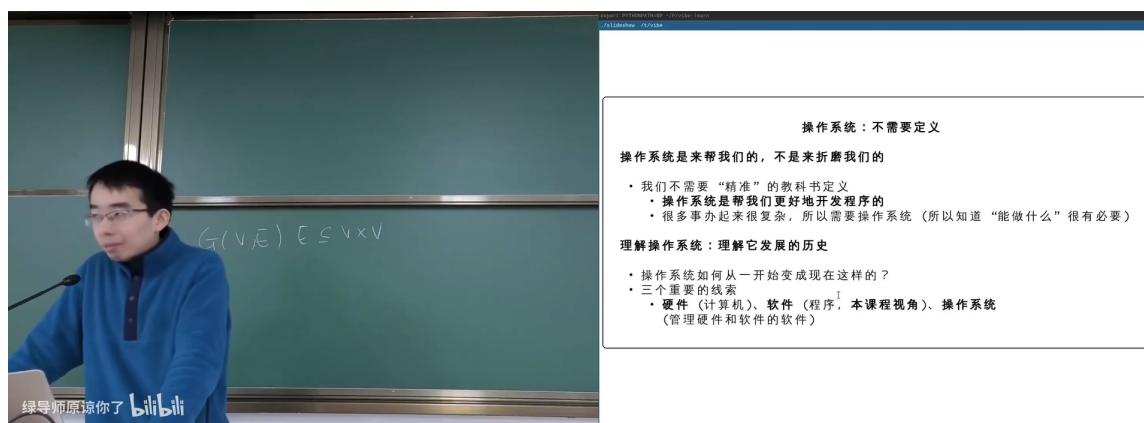


图 9: 操作系统是来帮我们的，不是来折磨我们的⁹

操作系统的直觉理解

“我们不需要‘精确’的教科书定义。操作系统是来帮我们的，不是来折磨我们的。”

理解操作系统：理解它发展的历史。

操作系统如何从一开始就变成现在这样的？三个要素构成了理解的基础：

- **硬件**（计算机的物理实现）
- **软件**（程序：本质程序/应用程序）
- **操作系统**（管理硬件和软件的软件）

3.2 复习：理解计算机硬件（电路）

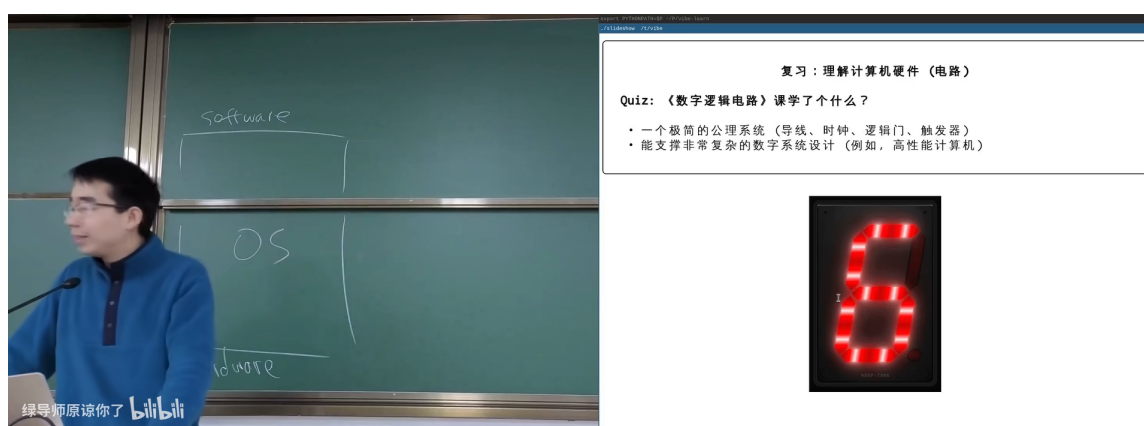


图 10: Quiz：数字逻辑电路教了什么？七段数码管演示¹⁰

jyy 通过七段数码管的例子回顾了《数字逻辑电路》课程的核心：

- 一个最简单的公理系统：导线、时钟、逻辑门、触发器

⁹视频画面时间区间：00:39:00–00:39:45。

¹⁰视频画面时间区间：00:43:00–00:44:00。

- 随后推导出有意义的数字系统设计（纯组合、高性能化计算）

从电路到操作系统的桥梁

计算机硬件的本质是数字逻辑电路的组合。理解了“导线 + 逻辑门 + 时钟 = 计算”，就为理解操作系统如何管理硬件奠定了基础。

3.3 Content as Code: 今年课程的革新

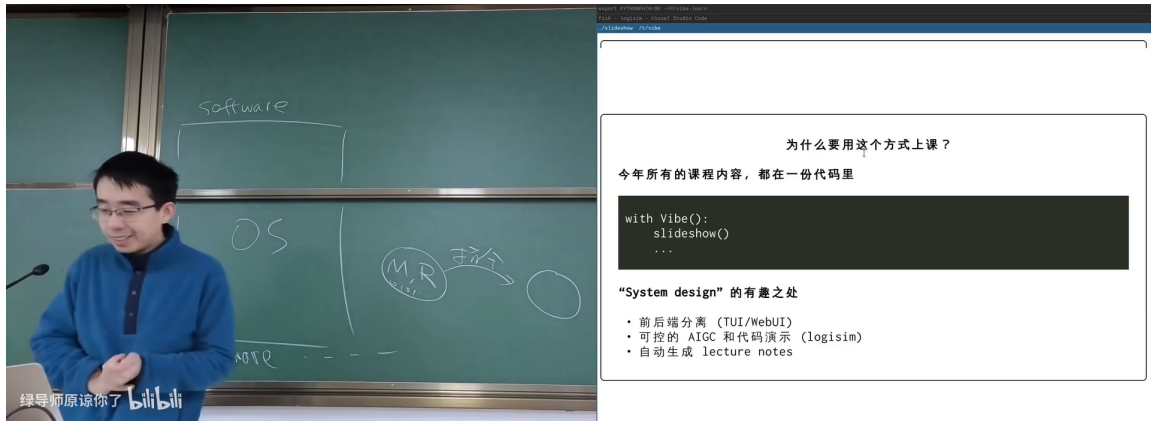


图 11: 为什么用代码驱动的方式上课: System Design 的有趣之处¹¹

2026 年课程的一个重大革新：所有课程内容都在一台代码里。

```
1 with Vibe():
2     slideshow()
3     ...
```

Listing 1: 课程内容代码化的核心理念

“System Design” 的有趣之处：

- 前后端分离 (TUI/WebGUI)
- 可控的 AIGC 和代码演示 (Logisim)
- 自动生成 lecture notes

¹¹ 视频画面时间区间：00:48:30–00:49:15。

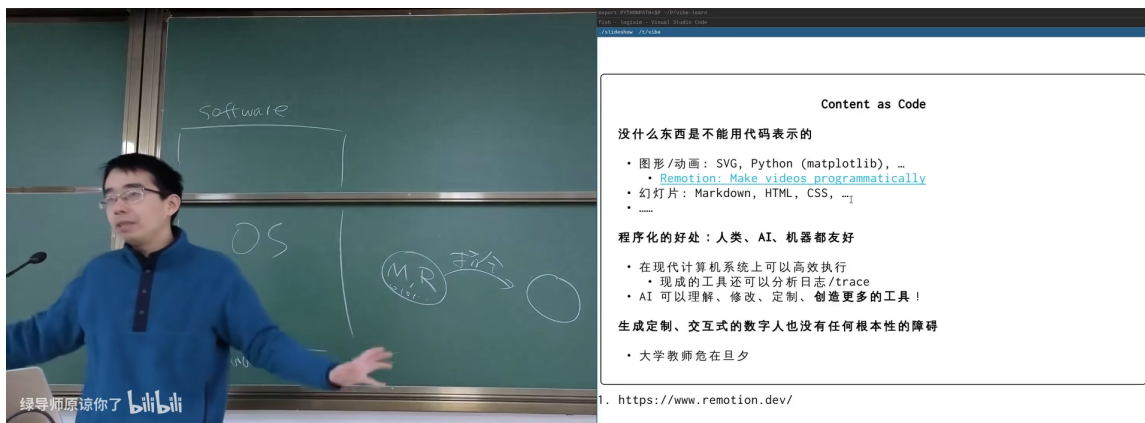


图 12: Content as Code: 什么东西不能用代码表示?¹²

Content as Code 的哲学

什么东西不能用代码表示的？几乎没有——

- 图表/动画：SVG、Python (matplotlib)..... 甚至可以 programmatically 生成
- 幻灯片：Markdown、HTML、CSS
- 程序化的好处：人类、AI、机器都友好

在现代计算机系统上可以执行任何代码！生成定制、交互式的数字人也没有根本性的障碍——大学数据就在那里。

3.4 复习：理解计算机软件（程序）

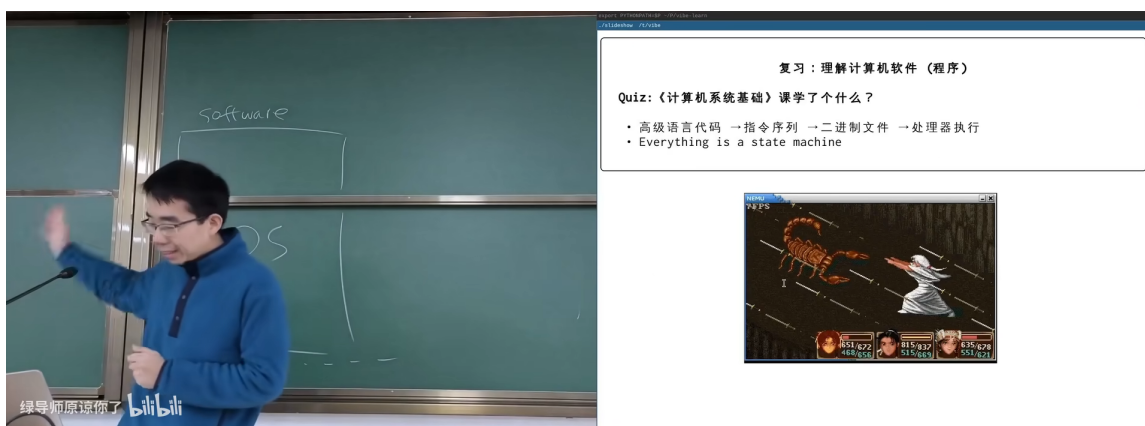


图 13: 计算机系统基础复习：Everything is a state machine¹³

《计算机系统基础》教了什么？

- 高级语言代码 → 指令序列 → 二进制文件 → 处理器执行

¹²视频画面时间区间：00:53:30–00:54:15。

¹³视频画面时间区间：00:58:30–00:59:15。

- **Everything is a state machine**——这是贯穿整个课程的核心思想

状态机视角的重要性

很多同学在学习操作系统时容易陷入 API 细节，而忘记了最根本的抽象：程序就是状态机，操作系统就是管理和调度这些状态机的超级状态机。如果你在后续学习中感到迷惑，回到这个视角往往能拨云见日。

3.5 本章小结

操作系统不需要一个精确的定义——它是硬件和软件之间的桥梁，是管理状态机的超级状态机。2026 年的课程采用“Content as Code”的全新方式，所有教学内容可程序化生成、可 AI 辅助、可交互展示。

4 操作系统的发展历史

jyy 用历史的视角串联了从 1940 年代到现代的操作系统演化。这不是简单的时间线罗列，而是通过每个时代的**硬件约束**来解释为什么操作系统会演化成现在的样子。

4.1 1940s：电子计算机的黎明

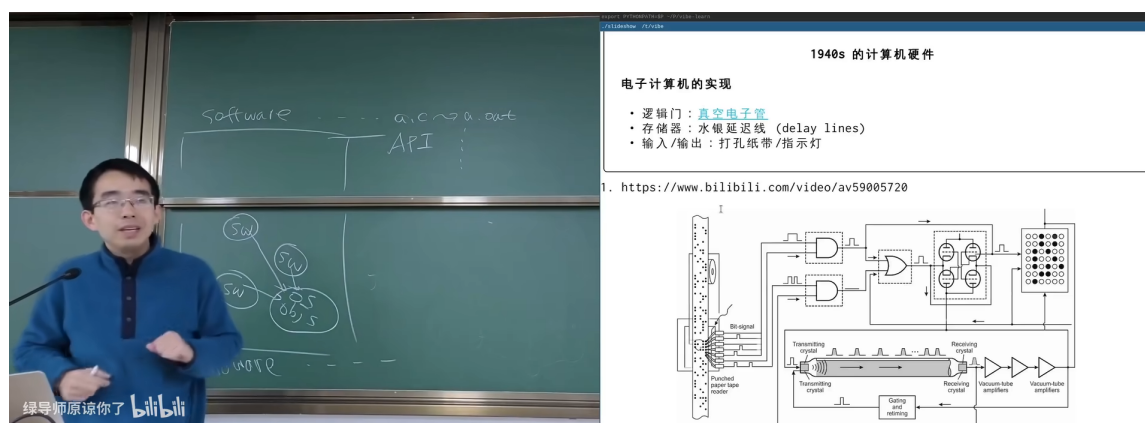


图 14: 1940 年代的计算机硬件：真空电子管、水银延迟线、打孔纸带¹⁴

1940 年代电子计算机的硬件实现：

- **逻辑门**：真空电子管（体积大、功耗高、可靠性低）
- **存储**：水银延迟线（delay lines）——利用声波在水银中的传播延迟存储数据
- **输入/输出**：打孔纸带/指示灯

¹⁴视频画面时间区间：01:06:30–01:07:15。

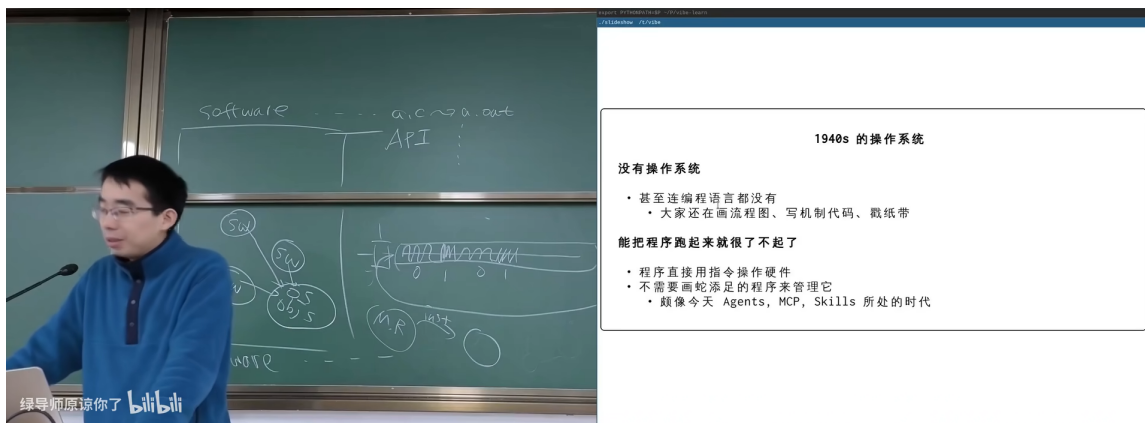


图 15: 1940 年代：没有操作系统¹⁵

为什么 1940s 不需要操作系统

程序直接操作硬件，根本没有编程语言——大家还在用高级组织代码。程序跑起来就够了，不需要高级组织来管理它。

jyy 做了一个有趣的类比：这就像今天的 Agents、MCP、Skills——**创世的时代**。一切都是直接的、原始的、没有中间抽象层的。

4.2 1950s–1960s: Fortran 与早期软件

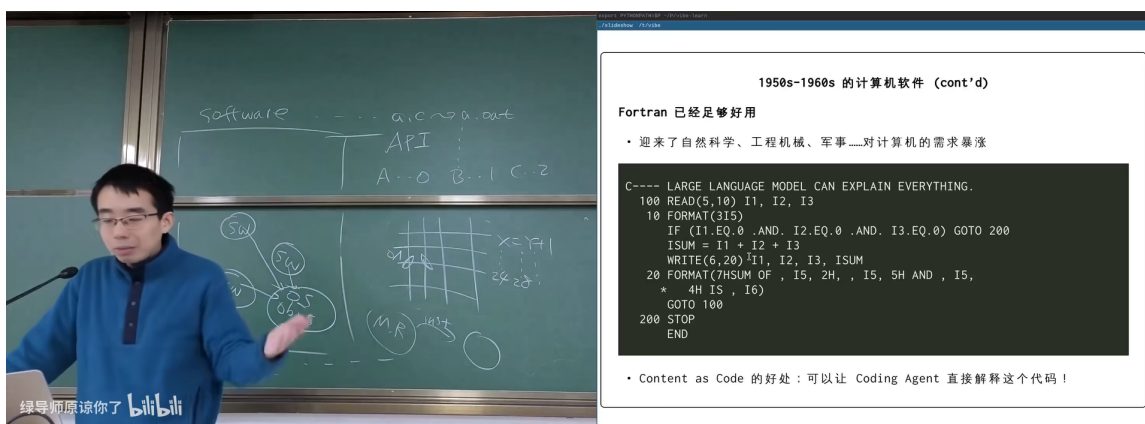


图 16: Fortran 代码示例：Content as Code 的好处——可以让 Coding Agent 直接解释！¹⁶

Fortran 出现后，计算机开始服务于自然科学、工程机械、军事的需求。jyy 展示了一段经典的 Fortran 代码，并插入了一句幽默的注释：

```
1 C----- LARGE LANGUAGE MODEL CAN EXPLAIN EVERYTHING.
2   100 READ(5,90) I1, I2, I3
3     90 FORMAT(3I5)
4     IF (I1.EQ.0 .AND. I2.EQ.0 .AND. I3.EQ.0) GOTO 200
5     ISUM = I1 + I2 + I3
6     WRITE(30,20) I1, I2, I3, ISUM
```

¹⁵ 视频画面时间区间：01:09:00–01:09:45。

¹⁶ 视频画面时间区间：01:14:30–01:15:15。

```

7      20  FORMAT(1X,I5,'_+_',I5,'_+_',I5,'_=_',I5,15X AND 15,
8      +      '_=_',A4, I5, I6)
9      GOTO 100
10     200  STOP
11     END

```

Listing 2: 经典 Fortran 代码 (含 jyy 的彩蛋注释)

Content as Code 的历史回响

jyy 指出: Content as Code 的好处在今天被放大了——可以让 Coding Agent 直接解释这些代码! 即使是 1950 年代的 Fortran, 现代 LLM 也能完美解读。这是“代码即内容”理念跨越 70 年的胜利。

4.3 1960s–1970s: 集成电路革命

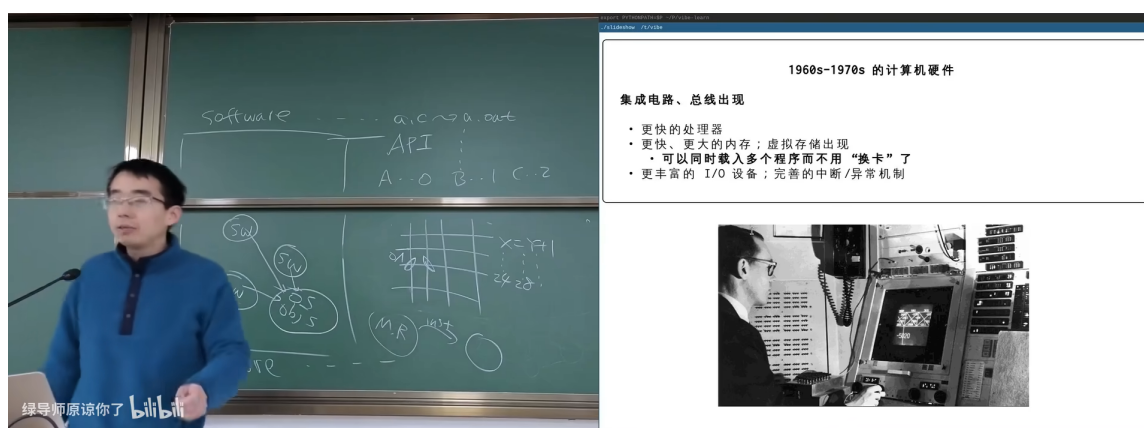


图 17: 集成电路与总线的出现: 操作系统的真正需求开始显现¹⁷

硬件的飞跃带来了操作系统的真正需求:

- 更快的处理器
- 更大的内存: 虚拟存储出现
- 多程序并发: 可以同时装入多个程序而不用“换卡”了
- 更丰富的 I/O: 显示器、键盘等交互设备出现

硬件驱动软件演化

操作系统不是某个天才凭空设计出来的——它是硬件能力的自然投射。当内存足够大到能同时容纳多个程序时,“如何调度”这个问题就自然出现了;当 I/O 设备变得多样时,“如何统一管理”就成了刚需。理解这一点,比记忆任何操作系统定义都重要。

¹⁷ 视频画面时间区间: 01:20:00–01:20:45。

4.4 1970s+：UNIX 与现代操作系统生态

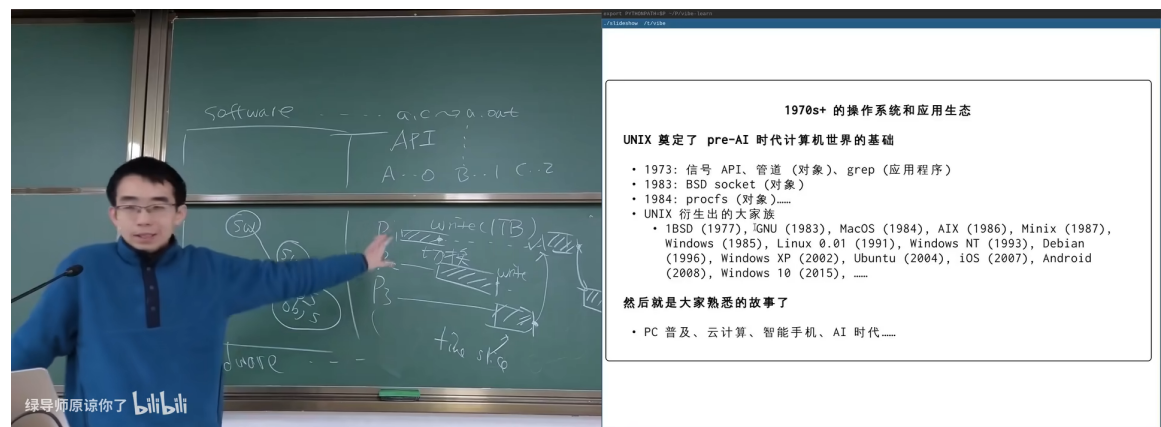


图 18: UNIX 奠定了 pre-AI 计算机世界的基础¹⁸

UNIX 是操作系统历史上最重要的里程碑。jyy 列出了关键的时间节点：

- **1973**：信号 API、管道（pipe）、grep（应用程序）
- **1983**：BSD socket（对象）
- **1984**：POSIX（对象?—“profile”）
- UNIX 的生态演化：BSD (1977) → GNU (1983) → MacOS (1984) → AIX (1986) → Minix (1987) → Windows (1985) → Linux 0.01 (1991) → Windows NT (1993) → Debian (1993) → Windows XP (2002) → Ubuntu (2004) → iOS (2007) → Android (2008) → Windows 10 (2015).....

UNIX 的遗产

UNIX 奠定了 pre-AI 时代计算机世界的基础。今天你使用的几乎每一个操作系统——macOS、Linux、Android、iOS——都可以追溯到 UNIX 的设计哲学：

- 一切皆文件
- 小工具组合完成复杂任务（管道）
- 层次化的文件系统
- 进程模型与信号机制

然后就是大家熟悉的故事了：PC 普及、云计算、智能手机、AI 时代.....

4.5 本章小结

操作系统的历史本质上是一部**硬件能力驱动软件抽象演化**的历史。从 1940s 的“程序直接操作硬件”到 UNIX 的“一切皆文件”，每一次抽象层的引入都是为了解决硬件能力增长带来的管理复杂性。这个模式在 AI 时代仍然在继续——Agent、MCP、Skills 就是新一代的“操作系统”概念。

¹⁸ 视频画面时间区间：01:28:00-01:29:00。

5 怎么学？

5.1 找到乐趣和动机

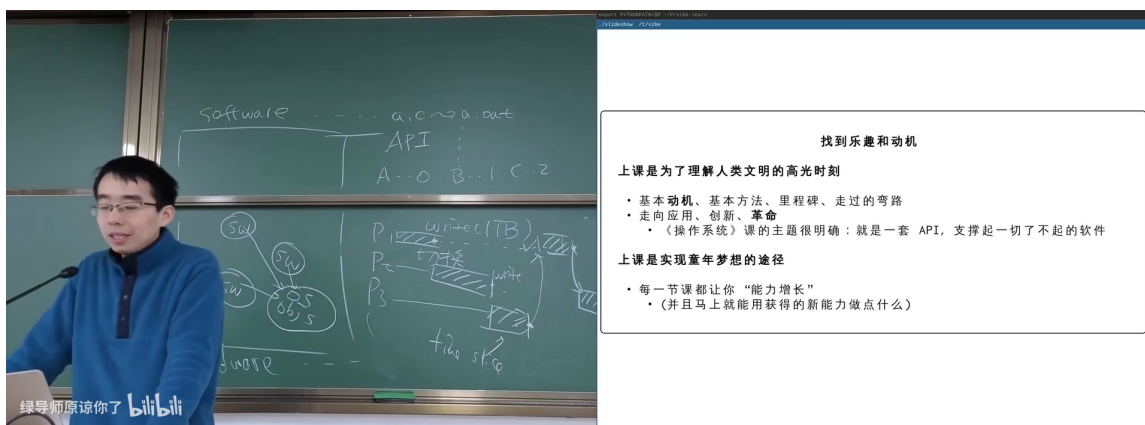


图 19: 上课的意义：理解人类文明的高光时刻¹⁹

jyy 的学习观：

- 上课是为了理解人类文明的高光时刻——基本概念、基本方法、算对数、走过的弯路，走向未来、创新、创业
- 操作系统课的主题明确：就是一套 API，支撑起一切了不起的软件
- 上课是实现美梦的途径：每一节课都让自己“能力提升”——你马上就能用新学到的能力做点什么

5.2 AIGC Policy

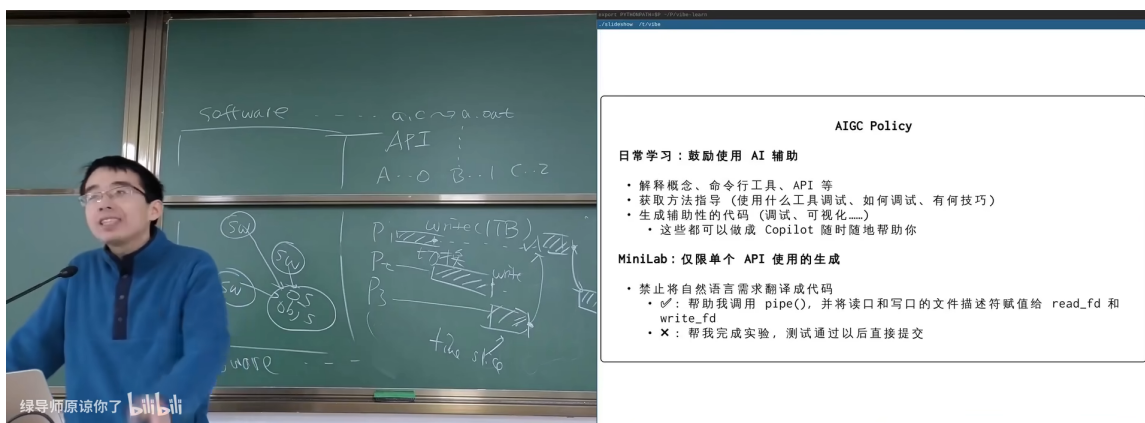


图 20: 日常学习鼓励使用 AI 辅助；MiniLab 的约束²⁰

¹⁹视频画面时间区间：01:33:00–01:33:45。

²⁰视频画面时间区间：01:35:00–01:35:45。

AIGC 使用政策

日常学习：鼓励使用 AI 辅助

- 解释概念、命令行工具、API 等
- 获取方法论（官方工具模式，如何调试，有何技巧）
- 获取知识（语法、逻辑、概念.....）
- 这些都可以用 Copilot 随时随地辅助你

MiniLab：仅限单个 API 使用的生成

- “星号系统挑战题要求你试代码”
- 外界条件改变程序的 `pipe()`，异常退出和窗口的文件描述符概念，`read_fd` 和 `write_fd`
- ×：基础完成实验，测试通过以后直接提交

5.3 本章小结

学习操作系统的关键不是死记硬背 API，而是找到乐趣——**理解系统如何被设计出来**，然后用 AI 工具加速实践。MiniLab 的设计精妙之处在于：限制你只用最基本的 API，迫使你真正理解底层原理。

5.4 拓展阅读

- jyy 的课程 Wiki: <https://jyywiki.cn/>
- StackOverflow: “How to exit Vim” (2017)——<https://stackoverflow.blog/2017/05/23/helping-one-million-developers-exit-vim/>
- “Stack Overflow is Dying” (2026)——搜索 “Stack Overflow is Dying blame AI coding tools”

6 总结与延伸

6.1 讲者的核心信息

蒋炎岩在本讲绪论中传递了一个清晰的信息：**Agentic AI 时代让操作系统课程变得更重要而非更不重要**。核心论证链条：

1. AI 能写代码，但不能替代对系统架构的理解
2. 操作系统是所有软件——包括 AI Agent——运行的基础
3. 理解操作系统 = 理解抽象层设计 = AI 时代最核心的人类能力
4. “Code is cheap, show me the talk”——理解比编码更重要

6.2 结构化提炼

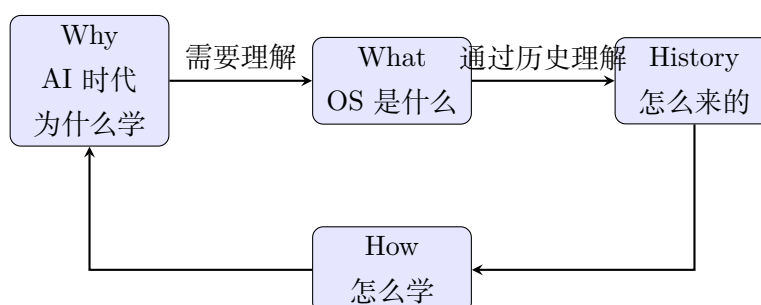
从本讲中可以提炼出以下核心洞察：

五个核心 Takeaway

1. **范式转移**：从“算法解决 well-formed 问题”到“AI 处理 ill-formed 问题”，人类的角色从“写代码”转向“设计抽象层”
2. **操作系统的本质**：不是 API 列表，而是“管理硬件和软件的软件”——一个由硬件约束自然演化出的抽象层
3. **Everything is a state machine**：程序是状态机，操作系统是管理状态机的超级状态机——这是贯穿整个课程的核心视角
4. **Content as Code**：2026 年课程本身就是“操作系统设计哲学”的实践——所有内容可程序化、可 AI 辅助、可自动生成
5. **历史驱动理解**：操作系统的每个设计决策都有历史原因——理解“为什么”比记住“是什么”更重要

6.3 跨章节关联

本讲的五个部分形成了一个紧密的逻辑闭环：



6.4 开放问题与下一步

1. AI Agent（如 Claude Code）本身需要什么操作系统支持？进程管理、文件系统、网络——这些都是后续课程的主题
2. “Content as Code”的极限在哪里？jyy 的课程本身就是一个实验
3. 当 AI 能通过 ICPC World Finals 时，计算机科学教育应该如何转型？这是一个超越操作系统课程本身的宏大问题